CSE312 Software Architecture: Principles and Practices

SDLC

- The benefits of software development life cycle approaches make your challenging and complex software development tasks smooth and easy.
- In fact, according to Zippia, businesses have experienced a 64% success rate using the Agile SDLC methodology while a 49% success rate using the Waterfall methodology.

What is the Software Development Life Cycle?

- The software development life cycle (SDLC) is a structured approach or framework that helps software development teams plan, design, develop, test, deploy, and maintain software applications.
- The primary goal of SDLC is to produce high-quality software that meets or exceeds customer expectations while staying within budget and on schedule.
- Traditionally, SDLC works on different methodologies to develop software, and the most common methodologies used by modern developers are Waterfall, Agile, and Scrum methodologies due to their flexible and iterative approaches.

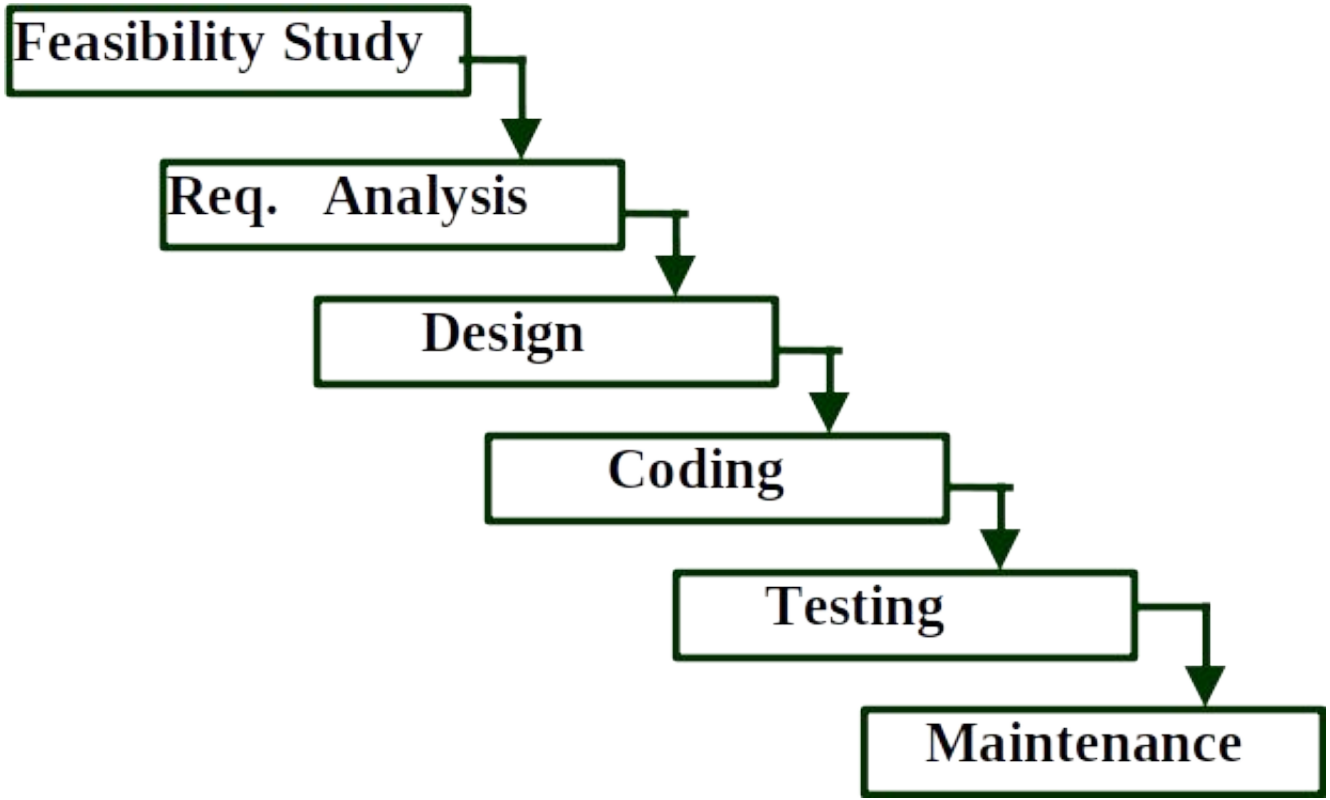
Advantages of SDLC

- Enhances Product Quality
- Improves Project Management
- Manages Risks Effectively
- Enhances Communication and Collaboration
- Enables Efficient Resource Utilization
- Flexibility and Adaptability

I. Classical Waterfall Model

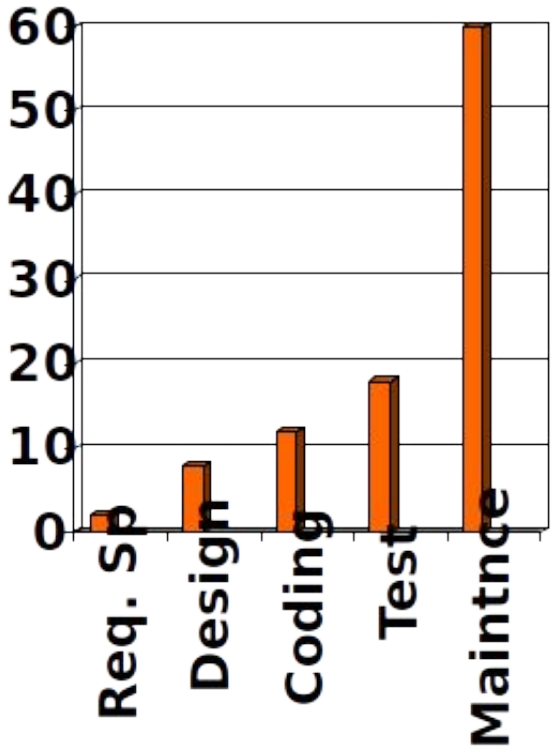
- Classical waterfall model divides life cycle into phases:
  - feasibility study,
  - requirements analysis and specification,
  - design,
  - coding and unit testing,
  - integration and system testing,
  - maintenance.

Classical Waterfall Model



Relative Effort for Phases

- Phases between feasibility study and testing
  - known as development phases.
- Among all life cycle phases
  - maintenance phase consumes maximum effort.
- Among development phases,
  - testing phase consumes the maximum effort.



Classical Waterfall Model (CONT.)

- Most organizations usually define:
  - standards on the outputs (deliverables) produced at the end of every phase
  - entry and exit criteria for every phase.
- They also prescribe specific methodologies for:
  - specification,
  - design,
  - testing,
  - project management, etc.

Classical Waterfall Model (CONT.)

- The guidelines and methodologies of an organization:
  - called the organization's software development methodology.
- Software development organizations:
  - expect fresh engineers to master the organization's software development methodology.

Feasibility Study

- Main aim of feasibility study:determine whether developing the product
  - financially worthwhile
  - technically feasible.
- First roughly understand what the customer wants:
  - different data which would be input to the system,
  - processing needed on these data,
  - output data to be produced by the system,
  - various constraints on the behavior of the system.

Activities during Feasibility Study

- Work out an overall understanding of the problem.
- Formulate different solution strategies.
- Examine alternate solution strategies in terms of:
  - resources required,
  - cost of development, and
  - development time
- Perform a cost/benefit analysis:
  - to determine which solution is the best.
  - you may determine that none of the solutions is feasible due to:
    - high cost, resource constraints, technical reasons.

Requirements Analysis and Specification

- Aim of this phase:
  - understand the exact requirements of the customer,
  - document them properly.
- Consists of two distinct activities:
  - requirements gathering and analysis
  - requirements specification

Goals of Requirements Analysis

- Collect all related data from the customer:
  - analyze the collected data to clearly understand what the customer wants,
  - find out any inconsistencies and incompleteness in the requirements,
  - resolve all inconsistencies and incompleteness.

Requirements Gathering

- Gathering relevant data:
  - usually collected from the end-users through interviews and discussions.
  - For example, for a business accounting software:
    - interview all the accountants of the organization to find out their requirements.

Requirements Analysis (CONT.)

- The data you initially collect from the users:
  - would usually contain several contradictions and ambiguities:
  - each user typically has only a partial and incomplete view of the system
- Ambiguities and contradictions:
  - must be identified
  - resolved by discussions with the customers.
- Next, requirements are organized:
  - into a Software Requirements Specification (SRS) document
- Engineers doing requirements analysis and specification:
  - are designated as analysts

Design

- Design phase transforms requirements specification:
  - into a form suitable for implementation in some programming language.
- In technical terms:
  - during design phase, software architecture is derived from the SRS document.
- Two design approaches:
  - traditional approach,
  - object oriented approach.

Traditional Design Approach

- Consists of two activities:
  - Structured analysis
  - Structured design

Structured Analysis Activity

- Identify all the functions to be performed.
- Identify data flow among the functions.
- Decompose each function recursively into sub-functions.
    - Identify data flow among the subfunctions as well.
- Carried out using Data flow diagrams (DFDs).
- After structured analysis, carry out structured design:
    - architectural design (or high-level design)
    - detailed design (or low-level design).

Structured Design

- High-level design:
  - decompose the system into modules,
  - represent invocation relationships among the modules.
- Detailed design:
  - different modules designed in greater detail:
    - data structures and algorithms for each module are designed.

Object Oriented Design

- First identify various objects (real world entities) occurring in the problem:
  - identify the relationships among the objects.
  - For example, the objects in a payroll software may be:
    - employees, managers, payroll register, Departments, etc.
- Object structure
  - further refined to obtain the detailed design.
- OOD has several advantages:
  - lower development effort,
  - lower development time,
  - better maintainability

Implementation

- Purpose of implementation phase (aka coding and unit testing phase):
  - translate software design into source code.
- During the implementation phase:
  - each module of the design is coded,
  - each module is unit tested
    - tested independently as a stand alone unit, and debugged,
  - each module is documented
- The purpose of unit testing:
  - test if individual modules work correctly.
- The end product of implementation phase:
  - a set of program modules that have been tested individually..

Integration and System Testing

- Different modules are integrated in a planned manner:
  - modules are almost never integrated in one shot.
  - Normally integration is carried out through a number of steps.
- During each integration step,
  - the partially integrated system is tested.



System Testing

- After all the modules have been successfully integrated and tested:
  - system testing is carried out.
- Goal of system testing:
  - ensure that the developed system functions according to its requirements as specified in the SRS document.

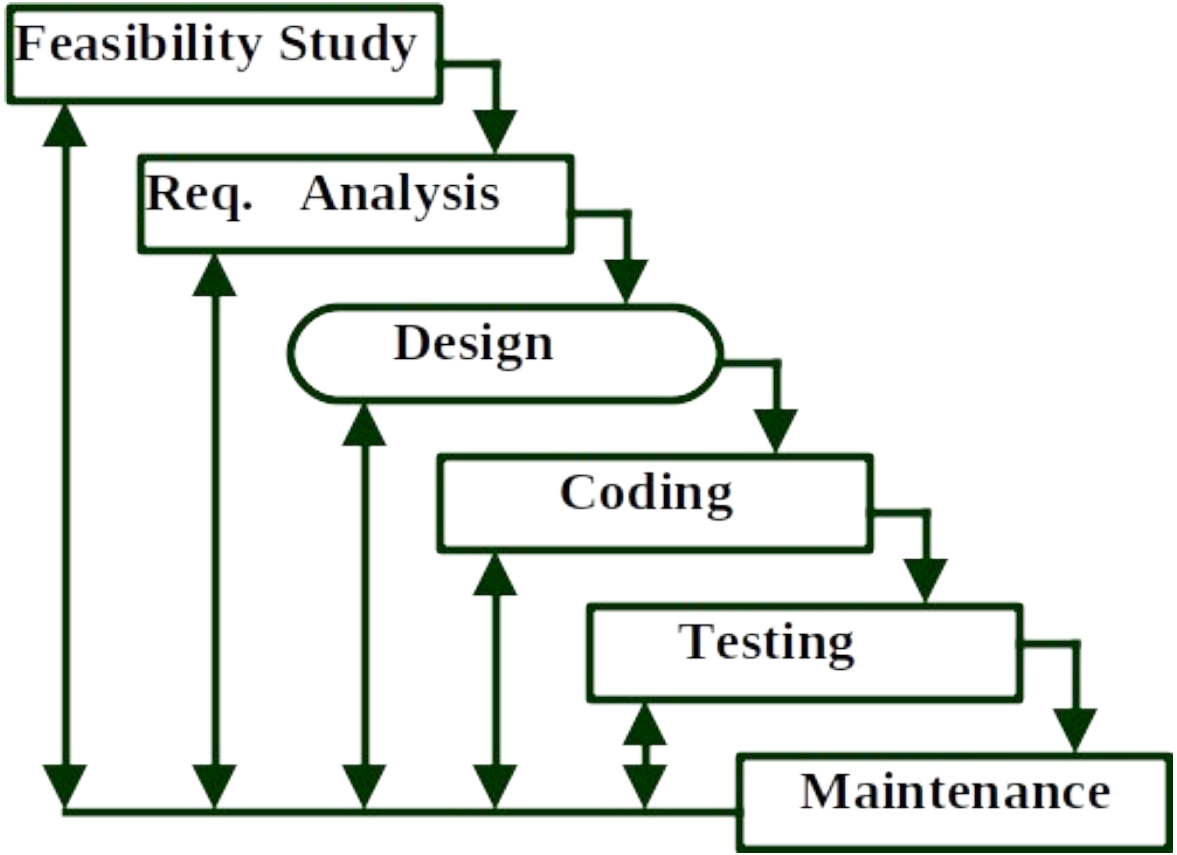
Maintenance

- Maintenance of any software product:
  - requires much more effort than the effort to develop the product itself.
  - development effort to maintenance effort is typically 40:60.
- Corrective maintenance:
  - Correct errors which were not discovered during the product development phases.
- Perfective maintenance:
  - Improve implementation of the system
  - enhance functionalities of the system.
- Adaptive maintenance:
  - Port software to a new environment,

II. Iterative Waterfall Model

- Classical waterfall model is idealistic:
  - assumes that no defect is introduced during any development activity.
  - in practice:
    - defects do get introduced in almost every phase of the life cycle.
- Defects usually get detected much later in the life cycle:
  - For example, a design defect might go unnoticed till the coding or testing phase.
- Once a defect is detected:
  - we need to go back to the phase where it was introduced
  - redo some of the work done during that and all subsequent phases.
- Therefore we need feedback paths in the classical waterfall model.

Iterative Waterfall Model (Cont.)



Iterative Waterfall Model (Cont.)

- Errors should be detected
  - in the same phase in which they are introduced.
- For example:
- if a design problem is detected in the design phase itself,
- the problem can be taken care of much more easily
- than say if it is identified at the end of the integration and system testing phase.

Phase containment of errors

- Reason: rework must be carried out not only to the design but also to code and test phases.
- The principle of detecting errors as close to its point of introduction as possible:
  - is known as phase containment of errors.
- Iterative waterfall model is by far the most widely used model.
- Almost every other model is derived from the waterfall model.

III. Prototyping Model

- Before starting actual development,
  - a working model of the system should first be built.
- A prototype is a toy implementation of a system:
  - limited functional capabilities,
  - low reliability,
  - inefficient performance.

Reasons for developing a prototype

- Illustrate to the customer:
  - input data formats, messages, reports, or interactive dialogs.
- Examine technical issues associated with product development:
  - Often major design decisions depend on issues like:
    - response time of a hardware controller,
    - efficiency of a sorting algorithm, etc.
- it is impossible to ``get it right'' the first time,
- we must plan to throw away the first product
  - if we want to develop a good product.

Prototyping Model (CONT.)

- Start with approximate requirements.
- Carry out a quick design.
- Prototype model is built using several short-cuts:
  - Short-cuts might involve using inefficient, inaccurate, or dummy functions.
    - A function may use a table look-up rather than performing the actual computations.
- The developed prototype is submitted to the customer for his evaluation:
  - Based on the user feedback, requirements are refined.
  - This cycle continues until the user approves the prototype.
- The actual system is developed using the classical waterfall approach

Prototyping Model (CONT.)

Step 1: customer idea

Step 2: build prototype

Step 3: customer feedback

Step 4: improve prototype

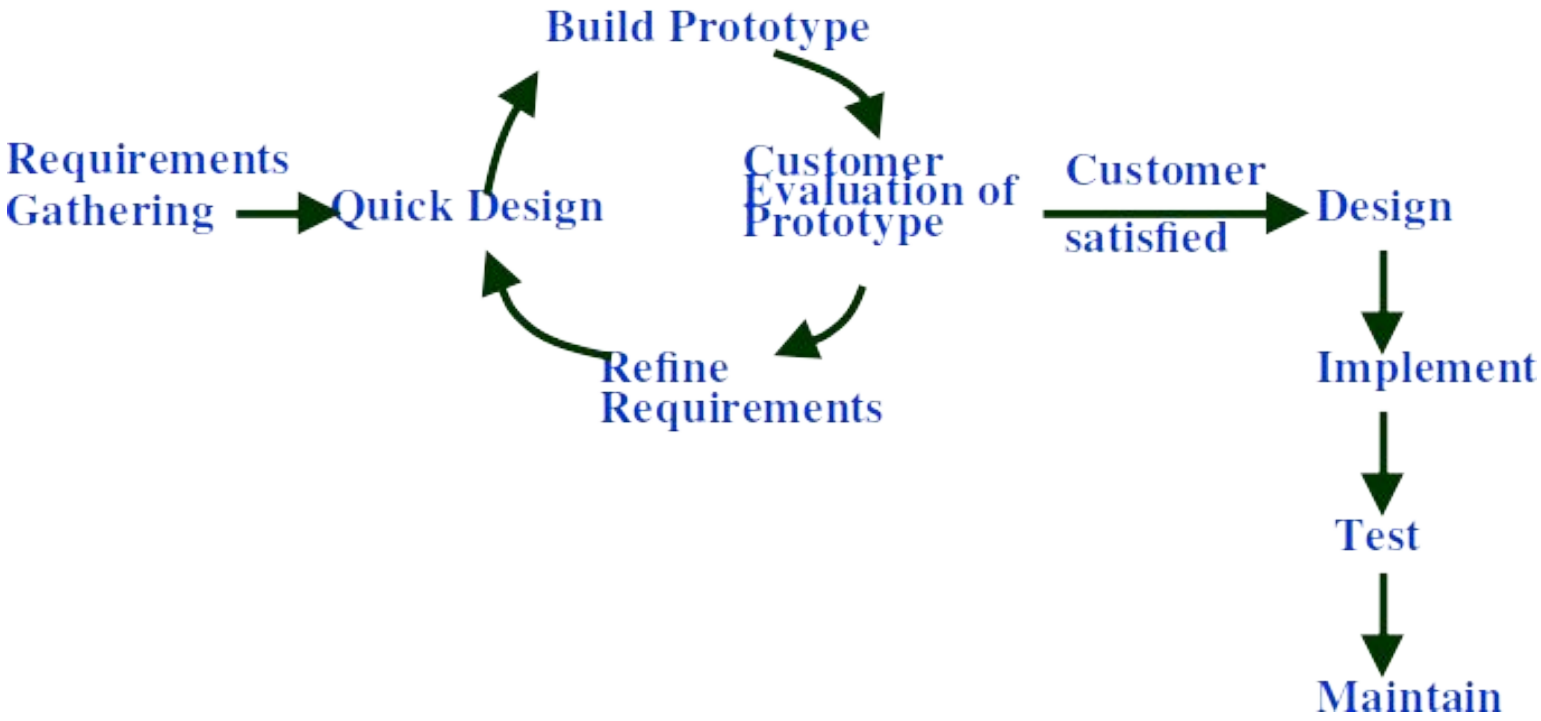
Step 5: customer feedback

Step 6: improve prototype

Step 7: final requirements

Step 8: actual software development

Prototyping Model (CONT.)



Prototyping Model (CONT.)

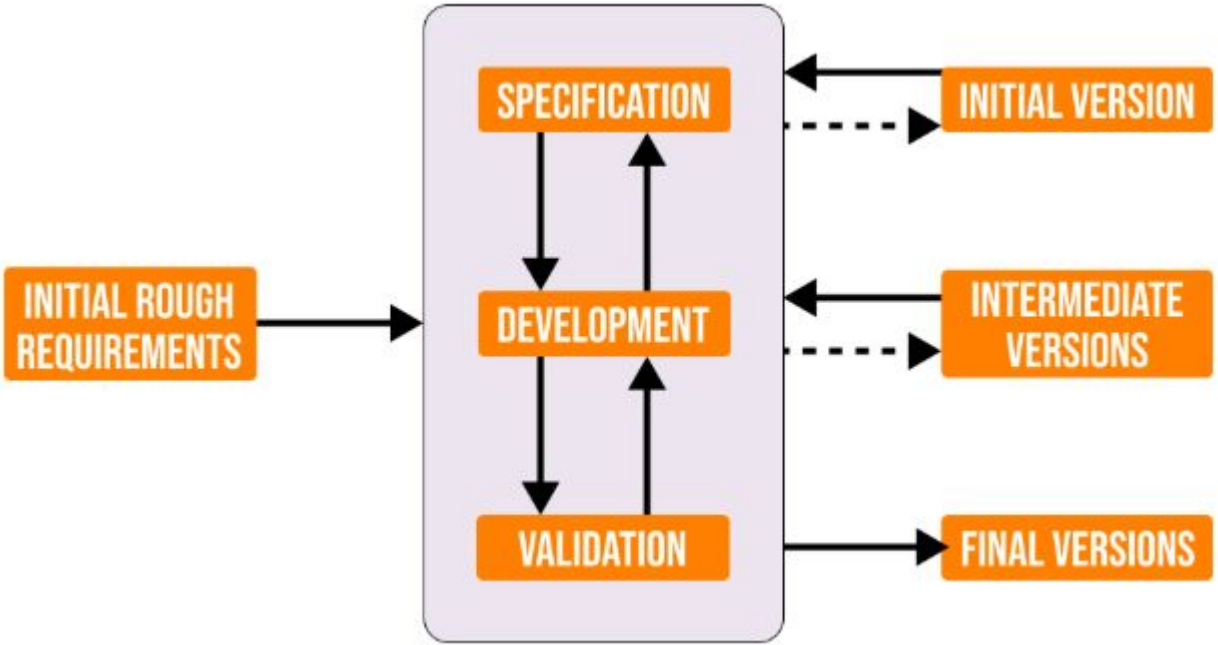
- Requirements analysis and specification phase becomes redundant:
  - final working prototype (with all user feedbacks incorporated) serves as an animated requirements specification.
- Design and code for the prototype is usually thrown away:
  - However, the experience gathered from developing the prototype helps a great deal while developing the actual product.

Prototyping Model (CONT.)

- Even though construction of a working prototype model involves additional cost --- overall development cost might be lower for:
  - systems with unclear user requirements,
  - systems with unresolved technical issues.
- Many user requirements get properly defined and technical issues get resolved:
  - these would have appeared later as change requests and resulted in incurring massive redesign costs.

IV. Evolutionary Model

- The software is developed step-by-step and improved continuously based on user feedback. It combines the ideas of Iterative Model and Incremental Model.
- Develop a small working version first, get feedback from users, improve it, and repeat the process until the final software is ready.
- Eg;- online shopping application
- Version 1 (Basic Product)
- First, developers create only essential features:
    - User registration,Login,View products,Search products
- The application is released to a small group of users.


SPECIFICATION
DEVELOPMENT
VALIDATION
INITIAL ROUGH REQUIREMENTS
INITIAL VERSION
INTERMEDIATE VERSIONS
FINAL VERSIONS

Types of Evolutionary Process Models

Iterative Model

- The same software is repeatedly improved through several cycles.
- Example:

  A mobile banking app:

Version 1 - Check balance

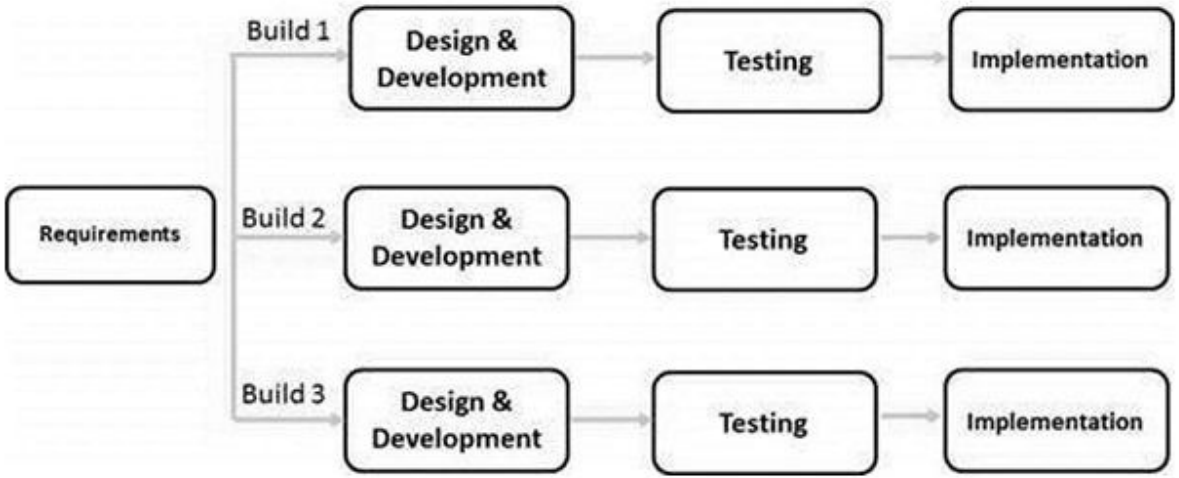
Version 2 - Add money transfer

Version 3 - Add loan services

Version 4 - Add investment features

- Each iteration improves the existing system.

Iterative Model



Incremental Model

- The software is divided into small parts (increments), and each part is developed separately.
- Example: College Management System

Increment 1:

Student registration

Increment 2:

Attendance management

Increment 3:

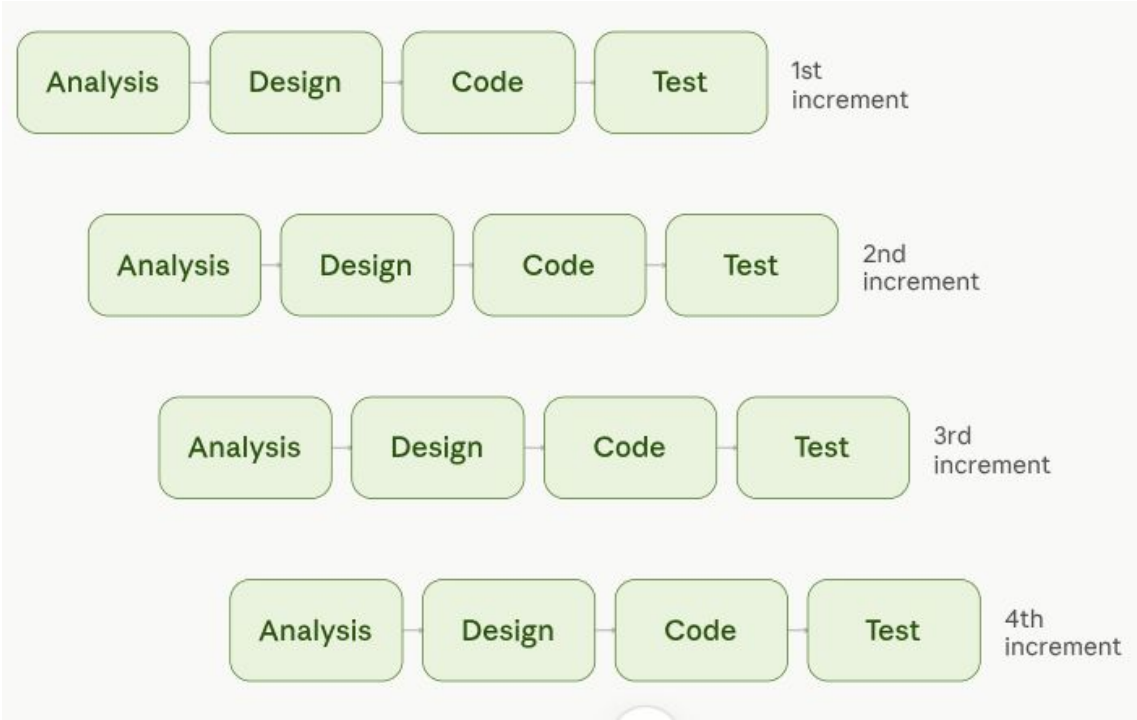
Examination management

Increment 4:

Fee payment system

- Finally, all parts are combined into one complete system

Incremental Model



Prototype vs Evolutionary Model

Prototype Model	Evolutionary Model
Build a rough model	Build a working product
Used to understand requirements	Used to add features gradually
Prototype is usually discarded or heavily reworked	Every version becomes part of the final software
Goal: Gather feedback	Goal: Deliver software incrementally

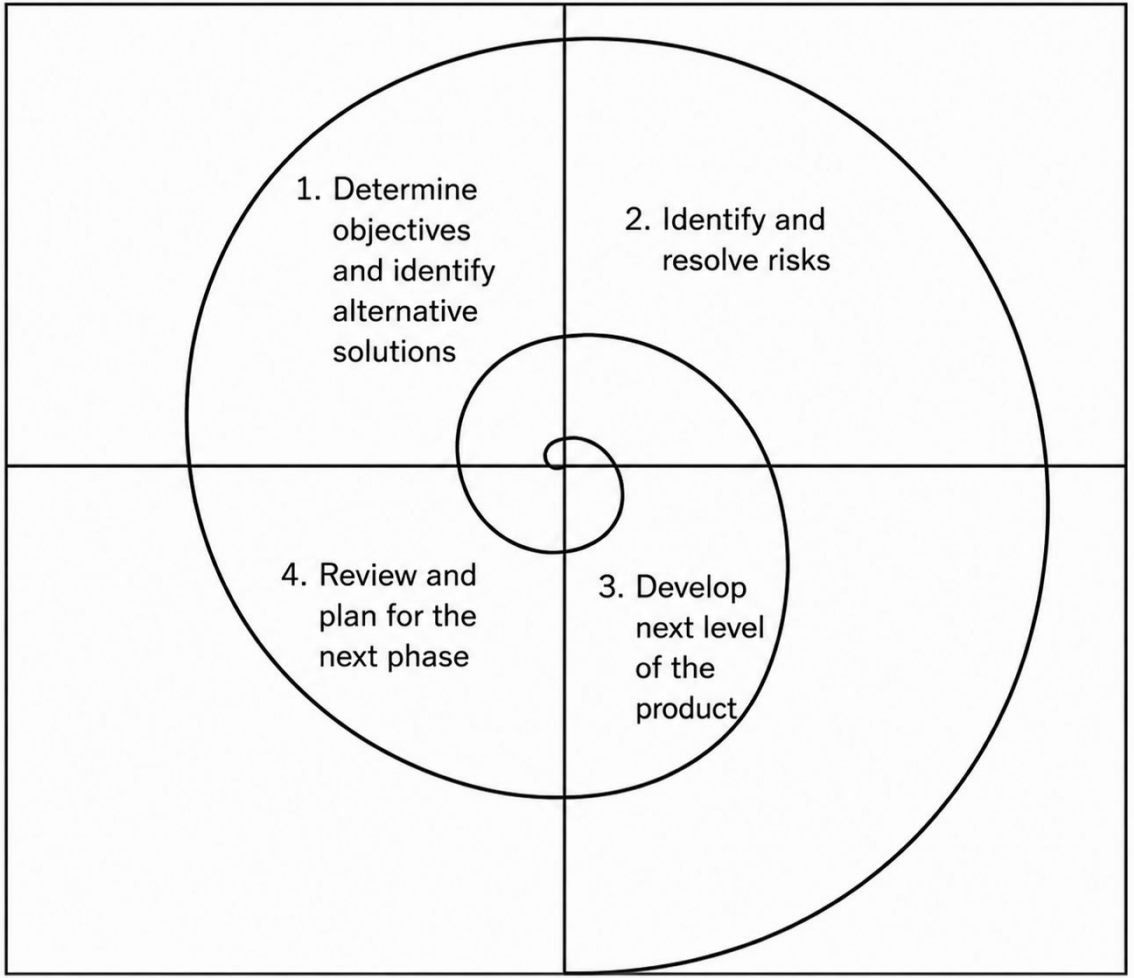
V. Spiral Model

- Proposed by Boehm in 1988.
- Each loop of the spiral represents a phase of the software process:
  - the innermost loop might be concerned with system feasibility,
  - the next loop with system requirements definition,
  - the next one with system design, and so on.
- There are no fixed phases in this model, the phases shown in the figure are just examples

Spiral Model (Cont.)

- The team must decide:
  - how to structure the project into phases.
- Start work using some generic model:
  - add extra phases
    - for specific projects or when problems are identified during a project.
- Each loop in the spiral is split into four sectors (quadrants).

Spiral Model (Cont.)



Objective Setting (First Quadrant)

- Identify objectives of the phase,
- Examine the risks associated with these objectives.
- Risk:
  - any adverse circumstance that might hamper successful completion of a software project.
- Find alternate solutions possible.

Risk Assessment and Reduction (Second Quadrant)

- For each identified project risk,
  - a detailed analysis is carried out.
- Steps are taken to reduce the risk.
- For example, if there is a risk that the requirements are inappropriate:
  - a prototype system may be developed.

Spiral Model (Cont.)

- Development and Validation (Third quadrant):
  - develop and validate the next level of the product.
- Review and Planning (Fourth quadrant):
  - review the results achieved so far with the customer and plan the next iteration around the spiral.
- With each iteration around the spiral:
  - progressively more complete version of the software gets built.

Comparison of Different Life Cycle Models

- Iterative waterfall model
  - most widely used model.
  - But, suitable only for well-understood problems.
- Prototype model is suitable for projects not well understood:
  - user requirements
  - technical aspects
- Evolutionary model is suitable for large problems:
  - can be decomposed into a set of modules that can be incrementally implemented,
  - incremental delivery of the system is acceptable to the customer.
- The spiral model:
  - suitable for development of technically challenging software products that are subject to several kinds of risks.